

AETHERIA — Key Flows (Data, Level-Up, Business)

Step 2 · sequence + state diagrams · Mermaid blocks render in GitHub & VSCode

1. Auth Flow (email + OAuth)

```
sequenceDiagram
    actor U as User
    participant W as Web (Next.js)
    participant A as API/Auth
    participant DB as Postgres
    participant R as Redis
    U->>W: enter email/password OR click OAuth
    W->>A: POST /auth/login (or /oauth/callback)
    A->>DB: SELECT user, verify argon2id / OAuth subject
    A->>R: SETEX session refresh token (rotated)
    A-->>W: { accessToken (15m), refreshToken (30d, httpOnly) }
    W->>W: store in memory + httpOnly cookie
    W->>A: subsequent calls with Authorization
    Note over A,W: silent refresh on 401 via /auth/refresh
```

2. Save / Resume Flow (autosave + cloud-sync)

```
sequenceDiagram
    participant G as Game Loop
    participant L as IndexedDB (local)
    participant A as Save Service
    participant DB as Postgres
    G->>L: snapshot every state change
    Note over G,L: instant, offline-safe
    loop every 30s OR onIdle/onHide
        G->>A: PUT /save/auto (slot=0, payload, baseVersion)
        A->>DB: UPSERT save_states WHERE user, slot=0 AND base<=current
        alt conflict (server newer)
            A-->>G: 409 + remoteSnapshot
            G->>G: reconcile(local, remote) – server wins gameplay, client wins cosmetic
            G->>A: retry PUT
        else
            A-->>G: 200 OK
        end
    end
    end
    Note over U: on next launch, resume banner
    G->>A: GET /save/list
    A-->>G: [{slot, level, location, updatedAt}]
    G->>A: POST /save/load (slot)
    A-->>G: payload
    G->>L: hydrate scene
```

3. Combat Turn Flow

```
sequenceDiagram
    participant C as Client
    participant API as API
    participant E as combat.applyAction (pure)
    participant DB as Postgres
    C->>API: action {runId, actor, type, target}
    API->>DB: SELECT run state + lastSeq
    API->>E: applyAction(state, action)
    E-->>API: {nextState, events[]}
```

```

API->>DB: APPEND action_log, UPDATE snapshot
API->>C: {events[], stateChecksum, seq+1}
C->>C: animate events; reconcile if checksum mismatch (rare)
Note over API: server is authoritative – client cannot cheat

```

4. Level-Up Flow (character & account)

```

flowchart TD
    Start[Action grants XP] --> A[progression.grantXp]
    A --> B{xp >= curveAt(level+1)?}
    B -- no --> persist1[persist[(persist xp)]]
    B -- yes --> Loop[loop levels gained]
    Loop --> C[stats += growthCurve]
    Loop --> D[unlock next skill node?]
    Loop --> E{milestone reached? lvl%5==0}
    E -- yes --> M[grant milestone reward]
    Loop --> F[emit LeveledUp event]
    F --> Q[quests.progress on event]
    Q --> N[notify.push]
    N --> persist

```

Curve: $x_p(n) = \text{floor}(50 * n^{1.85})$; max char level = 100. Milestone unlocks (account level):

- 5 → Photo Mode
- 10 → second character slot
- 15 → daily quests
- 25 → co-op raids
- 40 → ranked PvP
- 60 → guild creation
- 80 → endless tower
- 100 → epilogue + new+ mode

5. Matchmaking & Ranking Flow

```

sequenceDiagram
    participant C as Client
    participant Q as Match Queue (Redis)
    participant M as Matchmaker
    participant RT as Realtime
    participant DB as Postgres
    C->>Q: ZADD queue:{mode}:{region} score=mmr userId
    loop every 1s
        M->>Q: ZRANGEBYSCORE within ±bracket (widens over time)
        alt match found
            M->>DB: INSERT pvp_matches
            M->>RT: createRoom(matchId, players)
            RT-->>C: matchFound + room ticket
        end
    end
    end
    C->>RT: connect, subscribe match
    loop turns
        C->>RT: action
        RT->>RT: validate via combat engine
        RT-->>C: events broadcast
    end
    end
    RT->>DB: finalize match, write players, update mmr
    DB->>Redis: ZADD lb:{mode}:{season} mmrAfter userId

```

MMR algorithm: Glicko-2 (rating, deviation, volatility).

6. Quest Progress Flow (event-driven)

```
flowchart LR
    Action[Game event: enemyDefeated, levelCompleted, itemCrafted ...] --> Bus[Domain Event Bus]
    Bus --> QE[quests.progress evaluator]
    QE --> Match{matches active quest req?}
    Match -- yes --> Inc[increment progress jsonb]
    Inc --> Done{>= target?}
    Done -- yes --> Mark[status='completed']
    Mark --> Notif[notify.push 'quest ready']
    User[User clicks Claim] --> Claim[quests.claim]
    Claim --> Reward[inventory.grant + xp]
    Claim --> Audit[audit.write]
```

7. Guild Raid Flow (3-player co-op)

```
sequenceDiagram
    participant L as Guild leader
    participant API
    participant RT as Realtime
    participant P1 as Player1
    participant P2 as Player2
    L->>API: POST /guild/raid/schedule
    API-->>L: raidId, startsAt
    Note over P1,P2: at startsAt
    P1->>RT: join raidId
    P2->>RT: join raidId
    L->>RT: join raidId, ready
    RT-->>All: countdown
    loop turns (shared initiative)
        Active->>RT: action
        RT-->>All: events
    end
    RT->>API: finalize, distribute rewards
```

8. Marketplace Purchase Flow

```
sequenceDiagram
    C->>API: POST /shop/purchase (shopItemId, qty)
    API->>DB: BEGIN
    API->>DB: SELECT FOR UPDATE shop_items, profile.currency
    API->>DB: validate price, stock, currency
    API->>DB: INSERT transactions; UPDATE currency; INSERT inventory
    API->>DB: COMMIT
    API->>Audit: write transaction
    API-->>C: success + new balances
```

9. Anti-Cheat / Replay Validation

```
flowchart TD
    Run[Run completed] --> AC[worker.antiCheatScan]
    AC --> Replay[combat.replayActions(action_log)]
    Replay --> Check{state matches snapshot?}
    Check -- no --> Flag[mark suspicious, audit, throttle account]
    Check -- yes --> OK[clear]
```

10. Game State Machine (top-level)

```
stateDiagram-v2
    [*] --> Splash
    Splash --> Login
    Login --> MainMenu : authenticated
    MainMenu --> Continue : has save
    MainMenu --> NewJourney
    MainMenu --> Multiplayer
    MainMenu --> Codex
    MainMenu --> Shop
    MainMenu --> Settings
    Continue --> InGame
    NewJourney --> InGame
    InGame --> Combat
    InGame --> Exploration
    InGame --> Pause
    Pause --> InGame
    Pause --> MainMenu : exit (autosaves)
    Combat --> InGame : victory/defeat
    Multiplayer --> Queue
    Queue --> Match
    Match --> InGame : returns to hub
```